ARGUS Full-Stack Implementation Blueprint

Backend, Authentication, Shared Database, Automation Worker, Security and Deployment

ARGUS — SIH26262 Team

October 2026

ARGUS Full-Stack Implementation Blueprint

Purpose: This is an implementation contract for AI coding agents and human developers. It converts the remaining gaps into a concrete build plan: backend/API server, real authentication, server-side authorization, shared persistent database, durable automation processing, audit trail, tests and deployment instructions.

Product identity: ARGUS — AICTE Approval & Compliance Intelligence Platform
Problem statement: SIH26262 — Integrated AICTE Approval Workflow Automation, Compliance Monitoring and Alert Management System
Tagline: Complete Visibility. Accountable Governance.

ARGUS is a proposed hackathon solution, not an official or endorsed AICTE product.

> **Implementation directive:** Do not stop at describing an architecture or making the UI look complete. Inspect the current repository, implement the agreed full-stack vertical slice, run the tests, and report precisely what works and what remains blocked by missing credentials or deployment access.

1. Current-state truth and target state

Current reference prototype

The current prototype is a self-contained HTML/CSS/JavaScript browser demo using synthetic records and browser-local state. Its role selection and simulated notifications are useful for demonstrating the workflow concept but are not production security or shared persistence.

Unless the repository audit proves otherwise, assume these capabilities are still missing and verify each one:

- Real login/logout and server-validated user identity.
- Server-side role and record-scope authorization.
- A shared database that persists changes across browsers/users.
- A backend API which is authoritative for workflow transitions.
- A scheduled worker with durable run records, idempotency and failure visibility.
- Real audit persistence and atomic relationship between mutations and audit events.
- Private evidence/document storage and controlled download access.
- Staging/production deployment configuration, observability, backups and operational instructions.

Never infer that a capability exists from a button or static demo. Prove it through code, tests and a repeatable run.

Target for the next implementation

Build a **full-stack, multi-user hackathon MVP** with real authentication, shared PostgreSQL persistence, server-validated permissions, automation rules that can be rerun safely, an auditable workflow and a working frontend connected to the API. This is a production-shaped prototype, not a claim of production certification or permission to process real regulatory records.

2. Repository-first rule and default stack

Mandatory first step

The coding agent must inspect the existing repository before changing its framework. Identify the current frontend, backend, dependency manager, scripts, migrations, tests, hosting setup and environment variables. Preserve a working stack if one already exists. Do not delete or replace the current UI just to modernize it.

Recommended default only if no functioning backend/auth/database stack exists

- **Frontend:** retain the existing HTML/CSS/JavaScript prototype initially and split it into maintainable files only where useful. Use the existing framework if the repository already has one.
- **Backend API:** Python FastAPI, with typed request/response schemas and a service layer for workflow rules.
- **Identity provider:** Supabase Auth or another explicitly approved identity provider. Use real provider-managed sign-in/session handling, not a client-side role dropdown.

- **Shared database:** PostgreSQL, preferably the project's Supabase PostgreSQL instance for quick shared development. Use versioned migrations; SQLAlchemy/Alembic is the suggested Python path if suitable for the existing repository.
- **Evidence storage:** private object storage with scoped access; for a Supabase-based build, use a private Storage bucket and authorization-aware access policies.
- **Automation worker:** a separate worker process or managed scheduled job that evaluates due dates/rules. A development-only manual trigger may exist for testing, but it is not a substitute for the scheduled worker.
- **Notifications:** persisted in-app notifications first. External email/SMS delivery is an optional adapter and must be labelled simulated until provider credentials and delivery tests exist.

This is a pragmatic default, not an instruction to introduce every service regardless of need. If the current repo already has a working, secure equivalent, preserve it and record the decision in `docs/DECISIONS_AND_ASSUMPTIONS.md`.

3. Required architecture

The required logical separation is:

1. **Web client:** renders case lists, details, action forms, queues, deadlines, notifications and audit history. It never decides whether a user is authorized or whether a workflow transition is valid.
2. **Authentication:** provides login, logout, session refresh and verified identity claims. The client sends an access token to the API over HTTPS.
3. **API server:** verifies every token and validates each command. It is the authority for domain mutations and returns stable error codes.
4. **Authorization/policy layer:** resolves the user's active role, institution/region/case scope, allowed actions, route preconditions and relevant policy version.
5. **Domain services:** application routing, deficiency return/resubmission, EVC scheduling, compliance evidence verification, document renewal and audit/query services.
6. **PostgreSQL database:** stores users/profiles, scoped memberships, cases, route state, obligations, evidence metadata, rules, alerts, idempotency records, worker runs and audit events.
7. **Automation worker:** performs due/expiry scans, evaluates active rule versions, creates permitted tasks/alerts, tracks failures and retries safely.
8. **Notification outbox/adapter:** stores the intention to notify in the same transaction as the relevant state change, then delivers in-app or external notifications asynchronously.
9. **Private file storage:** keeps actual uploaded documents separate from relational metadata; authorization is checked before upload, download and review.

10. **Operations:** structured logs, request/correlation IDs, health checks, migration command, backup/restore instructions and visible failed jobs.

Workflow command path

```
HTTP request -> token verification -> role/scope authorization
-> input validation -> load current row/version -> validate state
transition and prerequisites -> evaluate configured rule/policy
version -> transactionally persist state + audit + idempotency/outbox
record -> worker/notification processing -> updated UI and traceable
audit
```

All important checks happen on the server, even if the frontend disables an action button. Do not trust browser-local state, a role field sent by the client, or a decoded-but-unverified token.

4. Authentication and authorization requirements

Authentication

- Implement login, logout, session restoration/refresh and a clear unauthenticated state.
- Verify the provider's access token on the server using the provider's documented verification method for the configured signing-key format. Validate signature, issuer, audience and expiry. Never merely base64-decode a JWT and treat its contents as true.
- Use invite/admin provisioning for privileged demo roles. Users cannot grant themselves an officer, reviewer, authority or administrator role through a signup form or local browser state.
- Provide synthetic development/test accounts through documented seed/admin setup. Do not place real passwords or production credentials in Git.
- Protect secrets with environment variables/secret management; keep server-only credentials out of frontend bundles and public files.

Authorization

Every API handler and every sensitive object operation must enforce both **action permissions** and **record scope**. A valid login alone is not sufficient.

Role	Allowed capabilities (subject to assigned scope)	Explicit restriction
Institution user	View own institution's cases/obligations; respond to deficiencies; submit evidence and renewal items	Cannot verify its own evidence, change policy or perform officer-only transitions
Scrutiny officer	Review assigned eligible cases; record observations; perform allowed scrutiny transitions; return for deficiency with reason	Cannot perform actions outside assigned scope or make decisions reserved to the competent authority
EVC coordinator/member	View eligible EVC queue; assign/schedule visits; track visit/report state	Cannot bypass route prerequisites or verify unrelated obligations
Compliance reviewer	Review submitted evidence and renewal items; record verification outcome and notes	Cannot verify evidence submitted by the same institutional identity where separation of duties applies
Regional/Bureau officer	Monitor permitted region/workload; assign follow-up; escalate according to policy	Cannot access other regions or edit global policy unless separately authorized
Competent authority	View complete authorized case file and record the reserved decision	The final decision must be an explicit human action with an audit trail, never an AI-generated automatic decision
Policy administrator	Manage route profiles/rule versions within granted scope; enable/disable configuration	Policy changes require validation, versioning and audit; admin role must not imply unlimited access to evidence content

The implementation may adapt role names to the existing canonical schema but must preserve the capability boundaries. Document any role/schema changes and update tests.

Tenant/record isolation

- Every institution-owned record must be scoped to an institution ID; region-bound roles must be scoped to authorized regions.
- Prevent insecure direct object reference (IDOR) bugs: changing an ID in a URL must not expose or mutate another tenant's records.
- Add database row-level security (RLS) or an equivalently verified database-level isolation strategy in addition to API authorization. Document exactly which paths use RLS and which checks are enforced in the server.
- Do not expose a service-role/admin database credential to the browser. If a server-only privileged credential bypasses RLS, the API must enforce authorization for every read/write and tests must prove isolation; prefer least-privileged DB roles and RLS wherever feasible.
- Return `403 Forbidden` for an authenticated user without permission, and avoid leaking sensitive existence/details when the caller has no access.

5. Shared database and schema requirements

Use PostgreSQL migrations under version control. Do not make the schema depend on manually clicking together tables in a dashboard. A new teammate must be able to create the schema from documented commands.

Minimum logical tables/entities

- `user_profiles`: auth-provider user ID, display metadata, account status, timestamps.
- `role_assignments` or `memberships`: role, institution/region scope, start/end dates, assigned-by and audit metadata.
- `institutions`: synthetic institution identifier and non-sensitive demo metadata.
- `applications`: institution, application type, route profile/version, current stage/status, owner/scope, due date, row version and timestamps.
- `route_profiles`, `route_stages` and `transition_rules`: configured paths and prerequisites; no universal hard-coded route.
- `stage_history`: from/to stage, actor/system identity, reason, event time, policy version and correlation ID.
- `deficiency_requests`: reason, missing items, stage to resume, status and response linkage.
- `evc_assignments` / `evc_events`: eligibility, coordinator, visit time, committee/panel metadata and report status.
- `compliance_obligations`: obligation type, source/policy reference, institution, due date, owner and state.
- `evidence_submissions`: private storage key, submitter, submitted-at time, review state, verifier, decision and notes.
- `documents` / `renewal_submissions`: document type, validity dates, storage key, submission/review status and review history.

- `automation_rules` and `automation_rule_versions`: enabled state, trigger, configured threshold, escalation route, source/effective dates, creator and approval metadata.
- `automation_runs` / `automation_run_items`: worker start/end/status, evaluated subjects, outcomes and failures.
- `idempotency_records`: unique key for command/event processing and result/reference.
- `alerts` / `notifications`: target user/role, trigger/rule version, severity, status, created/acknowledged/resolved times and delivery channel status.
- `notification_outbox`: durable send intent, destination reference, attempt count, next retry, status and last error; never store secrets in payloads.
- `audit_events`: append-only logical history with actor/system, action, entity, timestamp, outcome, reason, rule version, correlation ID and minimal safe metadata.

Do not duplicate unnecessary PII into the event log. Use foreign keys, indexes for common filters and unique constraints for idempotency. Use timestamps in UTC and present them in the user's timezone in the UI.

6. API and transaction contract

Use a versioned API namespace such as `/api/v1`. Exact framework syntax may vary, but behavior must be equivalent to this contract.

Required read endpoints

- `GET /health` — no sensitive configuration in the response.
- `GET /api/v1/me` — verified profile and permitted role/scope summary.
- `GET /api/v1/dashboard` — counts/aging derived only from the caller's authorized scope.
- `GET /api/v1/applications` and `GET /api/v1/applications/{id}`.
- `GET /api/v1/evc-queue` and `GET /api/v1/evc/{id}`.
- `GET /api/v1/compliance-obligations` and `GET /api/v1/evidence-submissions`.
- `GET /api/v1/documents` with expiry/renewal filters.
- `GET /api/v1/alerts` and `GET /api/v1/audit-events` with authorization and pagination.
- `GET /api/v1/rules` for allowed rule metadata; only policy administrators may read/edit restricted settings.

Required mutation endpoints

- Create intake/application; server chooses the configured initial state.
- Complete a stage; server validates role, current row version, route prerequisites and allowed next stage.
- Return for deficiency; requires a non-empty reason and persists the stage to resume.

- Resubmit response/evidence; returns to the designated review point, not an arbitrary client-selected stage.
- Schedule EVC and submit/record report separately.
- Submit evidence; resulting state is `Pending Verification`.
- Verify/reject/request clarification; only an authorized, non-conflicted reviewer may decide.
- Submit renewal and verify renewal as separate operations.
- Run an automation cycle through a test/admin-only trigger in development; production schedule runs through the worker identity.
- Acknowledge an alert without implying the underlying obligation is resolved.
- Create a new rule version through a privileged, validated and audited operation.

API rules

- Use typed schema validation and stable error codes: `VALIDATION_ERROR`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT_VERSION`, `INVALID_TRANSITION`, `PREREQUISITE_NOT_MET`, `DUPLICATE_ACTION`, `POLICY_INVALID`, `RATE_LIMITED`, `TEMPORARY_UNAVAILABLE`.
- Use optimistic concurrency (`row_version`/expected version) for case mutations.
- State-changing operations must not be implemented as `GET` requests.
- Mutations must persist domain state and audit information in one database transaction. Use an outbox for asynchronous messages.
- Return a request/correlation ID on success and error. Do not return raw stack traces or secrets to the client.
- Apply pagination and server-side filters; do not download every tenant's case list to the browser and filter locally.

7. Automation worker, reminders and notifications

Worker behavior

- Implement a separate callable worker/cron job that evaluates due dates and active rule versions on an explicit schedule.
- Persist each run and per-subject result. A failed item must be visible, logged with a correlation ID and retryable where safe.
- Generate an idempotency key with the subject, rule ID, threshold window and rule/policy version. Enforce uniqueness in the database; an in-memory set is not enough.
- Acquire work safely under concurrent workers (row locks/atomic claims or an equivalent design). Two worker processes must not create duplicate actions.
- Store the alert/task/audit/outbox intent before attempting external delivery. Retry transient failures with bounded backoff; report exhausted retries in a failed-job surface.

- Keep the development-only manual run endpoint behind authentication and an admin/demo permission. Production schedule must not depend on a human clicking a button.
- Avoid hard-coded legal timeline assumptions. Configured demo thresholds must say they are illustrative until validated against currently applicable official rules and authorized procedures.

Notifications

- P0: real database-backed in-app alerts that appear for the correct authorized user/role and persist after reload/login from another browser.
- Optional: email/SMS adapter only after provider configuration, secret handling, error/retry tracking and a real delivery test. Until then, label external delivery as simulated.
- Record delivery outcomes separately from the underlying alert. An email failure must not roll back an already persisted case transition.

8. Evidence and document storage

- Store metadata in PostgreSQL and file bytes in a private object store, not as public URLs.
- Validate permitted file types, size limits and metadata; do not trust filename extensions or client MIME declarations alone.
- Generate short-lived signed access links or stream files through an authorized endpoint after checking identity and scope.
- Log upload, download, review and export activity without logging the full document contents.
- Use synthetic, non-sensitive evidence in the hackathon. Malware scanning, retention controls and formal document authenticity checks are separate tasks; do not claim they exist unless implemented and tested.

9. Production-shaped security baseline

Before the app can be described as ready for real data, address at minimum:

- HTTPS at every deployed boundary and exact allowed CORS origins.
- Environment-based configuration and a `.env.example`; real secrets only in secret storage.
- No hard-coded credentials, service keys or personal data in Git.
- Authentication token validation, session expiry and logout behavior.
- Server-side action/record-scope authorization and tenant isolation tests for every role.
- Request validation, output encoding, rate limiting and safe error messages.
- CSRF/session protections appropriate to the chosen auth mode.
- Safe file upload/download and private storage.
- Database backups and a demonstrated restore procedure before production use.

- Dependency scanning/updates, structured logs, failed worker visibility and health checks.
- Admin/policy change audit, access review, incident-response owner and defined retention policy.

This baseline is engineering guidance, not security certification or a legal compliance opinion.

10. Deployment and configuration

Provide a `docker-compose.yml` or equivalent reproducible local environment if practical, plus separate instructions for local development and hosted staging. Keep hosting details adaptable to team access; do not hard-code a provider into the application logic.

Minimum config documentation:

- `SUPABASE_URL` / auth issuer URL.
- Public/anonymous auth key for browser usage only if the chosen provider requires it.
- Server-only privileged credential, if required by the selected architecture, clearly labelled secret and never included in frontend bundles.
- `DATABASE_URL` or documented provider DB connection configuration.
- JWT/JWKS verification configuration as required by the auth provider.
- `APP_ENV`, `APP_BASE_URL`, `ALLOWED_ORIGINS` and `LOG_LEVEL`.
- Private document-storage configuration.
- Notification provider settings only if real delivery is implemented.

The `.env.example` must contain placeholders, never usable secrets. Document database migration, seed, API start, worker start, tests, frontend start, and staging deployment commands. Configure health checks, logs and alerts for API/worker failures. Treat migration rollback/backup risks explicitly.

11. Test and acceptance plan

No feature is complete because a route returns HTTP 200. Add tests around actual persistence, permissions and workflow outcomes.

Authentication and authorization

- Anonymous user cannot read protected data or mutate a case.
- Expired, malformed and invalidly signed tokens are rejected.
- User cannot choose/change their role via request payload or browser state.
- Institution A cannot view or modify Institution B's case, evidence, alerts or document.
- Region-limited officer cannot access a different region.
- Institution user cannot verify own evidence; only authorized reviewers can verify.

- Competent-authority decision endpoint rejects other roles.
- Admin route/rule changes require privileged permission and generate an audit event.

Workflow and persistence

- Valid route transition succeeds and persists after restart and in a second browser/session.
- Invalid transition and unmet prerequisite are blocked by the backend.
- Deficiency return requires a reason and resubmission returns to the correct review stage.
- EVC eligibility, scheduling, visit and report status are distinct.
- Evidence submit creates pending verification; verification requires an authorized actor and note/outcome.
- Document expiry produces one warning per configured window; renewal submission does not equal verification.
- Every mutation creates a correlated audit event transactionally.
- Concurrent requests cannot silently overwrite newer case state.

Automation and delivery

- Running the same automation cycle twice creates no duplicate alerts/actions.
- Two concurrent worker instances do not double-process the same idempotency key.
- Transient delivery failure is retried; exhausted retries are visible.
- Alert creation persists even if an external notification provider is unavailable.
- Rule version changes affect only the intended policy version/effective window.

Release gate

The team must demonstrate the flows with synthetic users of different roles, use two separate browser sessions, restart the services and prove that shared state remains consistent. Record the exact commands and results in `docs/VALIDATION_RECORD.md`. If credentials/hosting are unavailable, label the related gate `BLOCKED BY CONFIGURATION` rather than pretending it passed.

12. Recommended implementation order for the 24-hour sprint

This order prioritizes a connected vertical slice. Re-estimate after the repository audit; do not skip tests to maintain an unrealistic schedule.

1. **Audit and freeze contracts (Hour 0-1):** inspect stack/scripts, choose default stack only if necessary, record decisions, set env variable names,

agree on roles, state names, APIs and ownership.

2. **Shared DB (Hours 1-4):** define migrations, foreign keys, indexes, role/scope tables, core workflow tables, audit/idempotency/outbox tables and synthetic seed data. Verify a clean reset/migration.
3. **Authentication (Hours 3-6):** provider login/logout/session, verified API identity, protected route and role loading; remove or prominently isolate fake role-switching from the authenticated experience.
4. **Server-authoritative workflow (Hours 5-10):** route-aware advance, invalid transition blocking, deficiency return/resubmit/resume, version conflict and transactional audit.
5. **EVC/compliance/document APIs (Hours 8-13):** scheduling/report status, evidence submit/verify separation and renewal/expiry lifecycle.
6. **Automation and in-app alerts (Hours 11-16):** scheduled worker/manual dev trigger, idempotency uniqueness, alerts, outbox and failure visibility.
7. **Connect frontend (Hours 14-19):** loading/error/empty states, real API calls, role-aware actions, cross-session persistence and honest integration status labels.
8. **Security/test pass (Hours 19-22):** permission matrix, tenant isolation, duplicate/concurrent run, boundary dates, migrations and build regression.
9. **Integration/rehearsal (Hours 22-24):** fresh setup, two-user scenario, final demo script, backup recording/screens, known-limits list and release notes.

If the shared backend/auth stack cannot be completed safely in the window, keep the static simulation clearly labelled and ship the backend work as an explicit branch or incomplete phase. Never present a non-integrated mock as real authentication or shared persistence.

13. Definition of done for coding agents

The implementation task is complete only when the agent provides:

1. Repository audit and stack decision (existing stack preserved or reasoned change).
2. Actual code changes for assigned scope, not documentation alone.
3. Database migrations and synthetic seed/reset instructions.
4. Authentication/token verification and server-side role/scope enforcement for protected operations.
5. The relevant API, UI and database wired together end-to-end.
6. Tests for positive, negative, unauthorized, cross-tenant, retry/idempotency and boundary-date paths where relevant.
7. Exact commands executed and real test/build outputs; no invented test success.
8. A concise list of implemented, unimplemented and credential-blocked capabilities.

9. Updated README, `.env.example`, API/architecture docs and validation record.
10. A runnable clean-setup path another teammate can follow without verbal instructions.

Required task-response format

- Objective and source requirements.
- Current repository finding.
- Files inspected/changed.
- Design and API/database contracts.
- Acceptance criteria.
- Commands run and exact results.
- Known gaps, configuration blockers and next integration owner.

14. Copy/paste execution prompt for a coding agent

You are implementing ARGUS, the SIH26262 approval workflow and compliance automation platform. Read `AGENTS.md`, `README.md`, `docs/PROJECT_CHARTER.md`, `docs/SOLUTION_SPECIFICATION.md`, `docs/24_HOUR_MVP.md`, this `docs/PRODUCTION_IMPLEMENTATION_BLUEPRINT.md`, and all relevant files under `technical/` before editing. First inspect the existing repo, scripts, dependencies, tests, environment variables and current prototype. Report which components actually exist and which are mock-only. Preserve a functioning stack unless the audit shows there is none; if no backend/auth/shared DB exists, use the recommended stack in this blueprint and record the decision. Then implement the first end-to-end slice: real provider-backed login, verified API identity, server-side role and record-scope authorization, a shared PostgreSQL schema via migrations, one server-authoritative application transition, a persisted audit event, a real database-backed in-app alert, and idempotent automation. Build in small commits; don't merely create more plans. Add migration/seed/setup commands, update `.env.example`, connect the frontend, and test anonymous/forbidden/cross-tenant/invalid-transition/duplicate-cycle cases. Continue to EVC, compliance evidence, document renewal, worker scheduling and deployment hardening in the order above. Do not expose server credentials, fake real email/SMS, invent legal timelines, or allow AI to make statutory decisions. At the end, report exact changed files and commands/results, plus what is blocked by missing credentials. Never call a simulated component production-ready.